# A kernel-checked bound $\Lambda \le 9/32$ for the de Bruijn–Newman constant from OpenAI's quasi-Riemann hypothesis

Peter W. Murphy

October 9, 2026; v1.1, October 12, 2026

Repository https://github.com/DrMurphyIsIn/qrh-debruijn-newman · DOI https://doi.org/10.5281/zenodo.23269135

**Abstract**

In October 2026 OpenAI released a Lean 4 proof that the Riemann zeta function has no zero with real part greater than 7/8. We compose that theorem with de Bruijn's 1950 heat-flow theorem, as formalized on the Arda project's `dbn` island, inside a single Lean environment pinned to OpenAI's toolchain and Mathlib revision. The result is the theorem

$$\forall\, t \ge \tfrac{9}{32}\ \ \forall\, z \in \mathbb{C}, \qquad H_t(z) = 0 \implies \operatorname{Im} z = 0,$$

which is the classical statement $\Lambda \le 9/32$ for the de Bruijn–Newman constant, and whose proof term depends only on `propext`, `Classical.choice` and `Quot.sound`. It is the first bound on $\Lambda$ below de Bruijn's 1/2 that is checked by a proof kernel end to end. It was re-checked by a second kernel (nanoda) through openai/ten-proofs Comparator against a challenge that imports only Mathlib, with a tampered challenge rejected; independently audited by a second session from a fresh checkout; and judged in continuous integration against a registered statement before it was granted. We also document the obstacle that had to be removed first: the two theorems were born under Mathlib revisions six weeks apart, and although the statement `riemannZeta s ≠ 0` is syntactically identical at both revisions, 267 definitions in its meaning closure differ. Composing across that seam would have been a mathematical identification rather than a kernel check; porting the de Bruijn machinery to OpenAI's revision (seventeen of nineteen modules byte-identical, two one-line proof repairs) removed the seam. The bound is weaker than the numerical bounds $\Lambda \le 0.22$ (Polymath15) and $\Lambda \le 0.2$ (Platt–Trudgian), and it is not the Riemann Hypothesis, which is $\Lambda = 0$. The mathematics is a one-line composition of two known theorems; the contribution is the verification, and this paper is its record. Version 1.1 adds, by the same converter, $\Lambda \le 2(B - \frac{1}{2})^2 = (1867057 - 10888\sqrt{921})/5464818 < 0.2811857$ from B. Liu's improvement of the half-plane to $\operatorname{Re} s > B = (1507 - 2\sqrt{921})/1653 \approx 0.874957$, which is the optimum of OpenAI's method with proved inputs (Section 6).

## 1 Introduction

### 1.1 The constant

Write $\xi$ for Riemann's entire function, $\xi(s) = \frac{1}{2}s(s-1)\pi^{-s/2}\Gamma(s/2)\zeta(s)$, whose zeros are exactly the nontrivial zeros of $\zeta$. Following the normalization of Rodgers and Tao [8] and of the Polymath15 project [9], put

$$H_t(z) \;=\; \int_0^\infty e^{tu^2}\,\Phi(u)\cos(zu)\,du, \qquad \Phi(u) \;=\; \sum_{n\ge 1}\bigl(2\pi^2 n^4 e^{9u} - 3\pi n^2 e^{5u}\bigr)\exp\bigl(-\pi n^2 e^{4u}\bigr). \quad (1)$$

Then $H_0(z) = \frac{1}{8}\,\xi\big(\frac{1}{2} + \frac{iz}{2}\big)$, so the Riemann Hypothesis is the statement that every zero of $H_0$ is real. The family $H_t$ is the evolution of $H_0$ under the backward heat equation $\partial_t H = -\partial_{zz} H$. De Bruijn [1] proved that if every zero of $H_{t_0}$ is real then so is every zero of $H_t$ for $t \geq t_0$, and that this is the case for $t_0 = 1/2$. Newman [2] proved that for some $t$ the zeros of $H_t$ are not all real, so that the threshold

$$\Lambda \;=\; \inf\{\, t \in \mathbb{R} : \text{every zero of } H_t \text{ is real} \,\}$$

is a well-defined real number, the de Bruijn–Newman constant, and he conjectured $\Lambda \geq 0$. Rodgers and Tao [8] proved Newman's conjecture in 2018. Together with de Bruijn's upper bound this places $\Lambda$ in $[0, 1/2]$, and the Riemann Hypothesis is exactly the statement $\Lambda = 0$.

The lower bounds have come from Lehmer pairs, consecutive zeros of $\zeta$ that lie anomalously close together, and they have converged on zero from below: $\Lambda > -50$ (Csordas, Norfolk and Varga [3]), successive improvements through Lehmer pairs (Csordas, Smith and Varga [4] and others), $\Lambda > -2.7 \times 10^{-9}$ (Odlyzko [5]), $\Lambda > -1.15 \times 10^{-11}$ (Saouter, Gourdon and Demichel [6]), and then Rodgers and Tao's $\Lambda \geq 0$. The upper bounds have moved more slowly. De Bruijn's $1/2$ stood for almost sixty years before Ki, Kim and Lee [7] made it strict. The Polymath15 project [9] proved $\Lambda \leq 0.22$ in 2019, and Platt and Trudgian [10] proved $\Lambda \leq 0.2$ in 2021. Both of the last two are computer-assisted: they combine rigorous numerical evaluation of $H_t$ at large height with analytic estimates, and the computations live outside any proof assistant.

### 1.2 The theorem

This paper records a bound that is far weaker than 0.2, and is the first one below $1/2$ to be a kernel-checked theorem in its entirety. In the sInf-free form that the formalization uses, because the island does not define $\Lambda$ (Theorem 5):

**Theorem 1.** *For every real $t \geq 9/32$ and every $z \in \mathbb{C}$, if $H_t(z) = 0$ then $\operatorname{Im} z = 0$. Classically, $\Lambda \leq 9/32 = 0.28125$.*

The proof is one line, and it is a composition of two theorems that are not ours. The first is OpenAI's.

**Theorem 2** (OpenAI, 2026 [12, 13])**.** *For every $s \in \mathbb{C}$ with $\operatorname{Re} s > 7/8$, $\zeta(s) \neq 0$.*

Theorem 2 is the headline of family 003, "The quasi-Riemann hypothesis", in OpenAI's October 2026 mathematics release [13]. It is formalized in Lean 4 against Mathlib's `riemannZeta` as `OAI.riemannZeta_ne_zero_of_seven_eighths_lt_re`, and the release ships a Comparator challenge for it whose statement imports only Mathlib. We replayed that challenge through Comparator with a second kernel before using the theorem (Section 5). Nothing in the present work proves anything about $\zeta$; we take Theorem 2 as it comes and we re-export it with its implicit binder closed.

A remark on what Theorem 2 is. Before it, every known zero-free region for $\zeta$ was of the de la Vallée Poussin shape $\operatorname{Re} s > 1 - c/\log(|\operatorname{Im} s| + 2)$ or an improvement of it: a region that hugs the line $\operatorname{Re} s = 1$ and thins out at height. No fixed vertical strip inside the critical strip had ever been shown free of zeros. A zero-free half-plane $\operatorname{Re} s > 7/8$ is a statement of a different kind, and by the functional equation it confines every nontrivial zero to $1/8 \leq \operatorname{Re} s \leq 7/8$. The Riemann Hypothesis is the limiting case $\operatorname{Re} s = 1/2$.

The second theorem is de Bruijn's, in the parametric form the Arda project formalized in September 2026 on its `dbn` island [14].

**Theorem 3** (de Bruijn 1950, Theorem 13; Polymath15, Theorem 3.2)**.** *Let $t_0, Y \in \mathbb{R}$. If every zero $z$ of $H_{t_0}$ satisfies $(\operatorname{Im} z)^2 \leq Y$, then for every $t \geq t_0$ every zero $z$ of $H_t$ satisfies $(\operatorname{Im} z)^2 \leq \max(Y - 2(t - t_0), 0)$.*

In particular, if the zeros of $H_0$ lie in the strip $|\operatorname{Im} z| \le y$, then the zeros of $H_t$ are real for $t \ge y^2/2$. A zero-free half-plane $\operatorname{Re} s > \theta$ for $\zeta$ puts the zeros of $H_0$ in the strip $|\operatorname{Im} z| \le 2\theta - 1$, through $H_0(z) = \xi(\frac{1}{2} + \frac{iz}{2})/8$ and the evenness of $H_0$, so:

**Corollary 4.** *If $\zeta(s) \neq 0$ whenever $\operatorname{Re} s > \theta$, then every zero of $H_t$ is real for every $t \ge 2(\theta - \frac{1}{2})^2$. Classically, $\Lambda \le 2(\theta - \frac{1}{2})^2$.*

At $\theta = 1$, the trivial half-plane, this is de Bruijn's $\Lambda \le 1/2$. At $\theta = 7/8$ it is $\Lambda \le 2 \cdot (3/8)^2 = 9/32$. Theorem 1 is Theorem 4 applied to Theorem 2.

### 1.3 What the contribution is, and what it is not

The composition is trivial as mathematics. What was not trivial was making the two theorems meet in one Lean environment, and then checking the result harder than it needed to be checked, so that the record of Theorem 1 is a record that can be trusted without trusting us.

The two theorems were born under different Mathlib revisions, six weeks apart. The Arda `dbn` island is pinned to Lean v4.34.0-rc1 and Mathlib `de5ce8a9`; OpenAI's library is pinned to Lean v4.34.1 and Mathlib `d13f23b`. Both state Theorem 2 with exactly the same characters, `∀ s : ℂ, (7 / 8 : ℝ) < s.re → riemannZeta s ≠ 0`, and a reader could be forgiven for thinking that the two statements mean the same thing by inspection. We exported both environments with lean4export and compared the meaning closure of the statement, every definition it unfolds to, after normalizing for renamed universes and hygienic names. The statements themselves are identical. But 267 named definitions beneath them differ, from Mathlib's reorganization of the algebraic hierarchy down to the core definition of `Bool.and`. No kernel run can check "OpenAI's $\zeta$ is our $\zeta$" across those revisions; the identification would be mathematics, not verification. Section 4 documents this.

So we removed the seam. The import closure of Theorem 4 on the `dbn` island is nineteen modules and 5,512 lines, plus two modules of the LiCriterion library that supply $\xi$ as an entire function. We copied all of it into OpenAI's workspace, at OpenAI's revision, and compiled it there. Seventeen of the nineteen modules compiled byte-identically. Two needed a one-line repair each, both to proof terms, both forced by a Mathlib rename or hypothesis change in the intervening six weeks, neither touching a statement or a definition. After that, Theorem 1 is one kernel term in one environment:

```
theorem dbn_real_zeros_of_qrh_unconditional :
    ∀ t : ℝ, 9 / 32 ≤ t → ∀ z : ℂ, DBN.H t z = 0 → z.im = 0 :=
  dbn_real_zeros_of_qrh qrh_seven_eighths
```

and `#print axioms` reports `[propext, Classical.choice, Quot.sound]`.

Then the verification. The theorem was re-checked by a second, independently written kernel through openai/ten-proofs Comparator [17], against a challenge file that imports only Mathlib and restates the definition of $H_t$ verbatim, so that the solution cannot substitute a different $H$; a tampered challenge, with one exponent in $\Phi$ changed, is rejected. A second session, from a fresh checkout of the pushed branch, verified OpenAI's statement and pins against GitHub, recomputed the import closure, diffed every ported file against its source, rebuilt everything and re-ran all three Comparator configurations with its own Comparator and nanoda builds. A continuous-integration job repeats the whole chain, cold, on every push. And the result entered a registry with a governance rule that neither the authoring session nor the auditing session could satisfy alone: the operator wrote the read-backs of the four formal statements under his own identity, the verify gate granted them, and a separate CI judge re-proved each registered statement from its artifact with both

kernels before the pass was recorded with the artifact hashed at the judged commit. Section 5 lays this out, with a table of what is kernel-checked and what is trusted as code.

What Theorem 1 does not do is move either frontier. It does not improve on 0.22, let alone 0.2. It does not bring the Riemann Hypothesis closer, because the conversion in Theorem 4 is lossy by its nature: a half-plane at $\frac{1}{2}+\varepsilon$ yields only $\Lambda \le 2\varepsilon^2$, and only $\varepsilon = 0$, which is RH itself, yields $\Lambda = 0$. Section 7 says this precisely, and records the one number from the exercise that is worth keeping: a zero-free half-plane $\operatorname{Re} s > \theta$ beats the Polymath15 bound through Theorem 4 exactly when $\theta < \frac{1}{2} + \sqrt{0.11} \approx 0.8317$. OpenAI's $7/8 = 0.875$ is just short.

**Organization.** Section 2 gives the formal statements exactly as checked. Section 3 gives the proof of Theorem 4 as the island carries it out. Section 4 is the cross-revision analysis and the port. Section 5 is the verification chain. Section 7 is the honest assessment. Section A is the reproduction recipe and Section B the verdicts, verbatim.

## 2 The formal statements

Everything in this section is a Lean 4 declaration in the repository that accompanies this paper [15], compiled in the materialized workspace described in Section A. We give the statements as checked and describe the objects in words. `DBN.H` is $H_t(z)$; `riemannZeta` is Mathlib's $\zeta$.

### 2.1 The heat-flow family

The `dbn` island defines the objects of Equation (1) directly (`DBNDefs.lean`):

```
noncomputable def Φ (u : ℝ) : ℝ :=
  ∑' n : ℕ+, (2 * Real.pi ^ 2 * (n : ℝ) ^ 4 * Real.exp (9 * u)
      - 3 * Real.pi * (n : ℝ) ^ 2 * Real.exp (5 * u))
    * Real.exp (-Real.pi * (n : ℝ) ^ 2 * Real.exp (4 * u))

noncomputable def HIntegrand (t : ℝ) (z : ℂ) (u : ℝ) : ℂ :=
  ((Real.exp (t * u ^ 2) : ℝ) : ℂ) * (((Φ u : ℝ) : ℂ) * Complex.cos (z * u)

noncomputable def H (t : ℝ) (z : ℂ) : ℂ := ∫ u in Set.Ioi (0 : ℝ), HIntegrand t z u
```

Here $\sum'$ is Mathlib's unconditional sum and the integral is the Bochner integral over $(0,\infty)$. These are the Polymath15 objects. The connection to $\xi$ is a theorem of the island, `dbn_H0_eq_xi`:

$$H_0(z) \;=\; \tfrac{1}{8}\cdot \xi\big(\tfrac{1}{2}+\tfrac{iz}{2}\big)\,,$$

where $\xi$ is LiCriterion's `riemannXi`, $\xi(s) = \frac{1}{2}s(s-1)\Lambda_0(s) + \frac{1}{2}$ in terms of Mathlib's $\texttt{completedRiemannZeta}_0$, the completed zeta function with its two poles removed. The $+\frac{1}{2}$ compensates for the pole removal: $\Lambda(s) = \Lambda_0(s) - 1/s - 1/(1-s)$, and $\frac{1}{2}s(s-1)\big(1/s + 1/(1-s)\big) = -\frac{1}{2}$. The theorem `riemannXi_eq_zero_iff_strip_zero` says that $\xi(s) = 0$ exactly when $\zeta(s) = 0$ and $0 < \operatorname{Re} s < 1$.

**Remark 5.** The island deliberately does not define $\Lambda$. A definition as an infimum over the reals would require the set $\{t : \text{all zeros of } H_t \text{ real}\}$ to be nonempty and bounded below before `sInf` means anything, and both facts are theorems in their own right (de Bruijn's upper bound and Rodgers–Tao's lower bound). Statements about $\Lambda$ are therefore carried in the equivalent "sInf-free" form: "every zero of $H_t$ is real for every $t \ge c$" stands for $\Lambda \le c$. Nothing in this paper depends on $\Lambda \ge 0$, and the registry does not record it.

## 2.2 De Bruijn's theorem

`DBNM1Parametric.lean` on the island proves Theorem 3 as

```
theorem dbn_debruijn_parametric :
    ∀ t0 Y : ℝ, (∀ z : ℂ, DBN.H t0 z = 0 → z.im ^ 2 ≤ Y) →
    ∀ t : ℝ, t0 ≤ t → ∀ z : ℂ, DBN.H t z = 0 → z.im ^ 2 ≤ max (Y - 2 * (t - t0)) 0
```

with no side condition on $t_0$ or $Y$. The proof goes through a Hadamard product representation of $H_{t_0}$ (order $< 2$), the heat-flow contraction of each factor, and Hurwitz's theorem for the limit, and it is the subject of the September 2026 Arda work recorded in the registry as `RH_dbn_debruijn_parametric`; this paper takes it as given. The specialization that Theorem 4 uses is

```
theorem m1_H_real_zeros_of_im_sq_le {t0 Y t : ℝ}
    (h0 : ∀ z : ℂ, H t0 z = 0 → z.im ^ 2 ≤ Y)
    (ht : t0 ≤ t) (hY : Y ≤ 2 * (t - t0)) {z : ℂ} (hz : H t z = 0) : z.im = 0
```

## 2.3 The corollary, conditional

`DBNZeroFreeHalfplane.lean`, added to the island on October 7, proves Theorem 4 for every real $\theta$:

```
theorem dbn_real_zeros_of_zeta_halfplane (θ : ℝ)
    (hfree : ∀ s : ℂ, θ < s.re → riemannZeta s ≠ 0) :
    ∀ t : ℝ, 2 * (θ - 1 / 2) ^ 2 ≤ t → ∀ z : ℂ, DBN.H t z = 0 → z.im = 0
```

and the instance $\theta = 7/8$, with the hypothesis written exactly as OpenAI's theorem with its implicit binder closed:

```
theorem dbn_real_zeros_of_qrh
    (hqrh : ∀ s : ℂ, (7 / 8 : ℝ) < s.re → riemannZeta s ≠ 0) :
    ∀ t : ℝ, 9 / 32 ≤ t → ∀ z : ℂ, DBN.H t z = 0 → z.im = 0
```

Both carry no assumption on $\theta$: for $\theta < 1/2$ the two one-sided bounds in the proof contradict each other, so $H_0$ has no zeros at all under the hypothesis and the conclusion is vacuously true. (By Hardy's theorem the hypothesis is false for $\theta < 1/2$; the formalization does not use this.)

## 2.4 The half-plane, re-exported

OpenAI's theorem, in `lean/OAI/NumberTheory/DirichletL/Nonvanishing.lean` of [13], reads

```
theorem riemannZeta_ne_zero_of_seven_eighths_lt_re
    {s : ℂ} (hs : (7 / 8 : ℝ) < s.re) : riemannZeta s ≠ 0 :=
  SevenEighths.ProbeFinalAssemblyUnconditional.zeta_nonzero s hs
```

Our bridge module `ArdaQRHBridge.lean` is compiled inside an unmodified checkout of OpenAI's `lean/` directory at commit `adc7f124`, and adds one declaration:

```
import OAI.NumberTheory.DirichletL.Nonvanishing

theorem qrh_seven_eighths : ∀ s : ℂ, (7 / 8 : ℝ) < s.re → riemannZeta s ≠ 0 :=
  fun _ hs ↦ OAI.riemannZeta_ne_zero_of_seven_eighths_lt_re hs
```

The proof is entirely OpenAI's. The file header, the repository's `NOTICE`, and this paragraph say so.

### 2.5 The theorem, unconditional

With the nineteen island modules ported into the same workspace (Section 4), `ArdaDBNUnconditional.lean` imports both sides and states Theorem 1:

```
import DBNZeroFreeHalfplane
import ArdaQRHBridge

theorem dbn_real_zeros_of_qrh_unconditional :
    ∀ t : ℝ, 9 / 32 ≤ t → ∀ z : ℂ, DBN.H t z = 0 → z.im = 0 :=
  dbn_real_zeros_of_qrh qrh_seven_eighths
```

Both occurrences of `riemannZeta`, the one inside `dbn_real_zeros_of_qrh`'s hypothesis and the one in `qrh_seven_eighths`'s conclusion, are now the same constant of the same environment, which is what makes the application typecheck without any identification step. The axiom guard `AxiomGuardDBNUnconditional.lean` prints the axioms of this theorem and of seven of its ancestors; all eight are `[propext, Classical.choice, Quot.sound]`.

## 3 The composition

The proof of Theorem 4 on the island is short enough to give in full, and it is worth giving because every step is a named island theorem that the registry judge has separately certified.

Fix $\theta$ and the half-plane hypothesis. Let $w$ be a zero of $H_0$. By `dbn_H0_eq_xi` and the factor $1/8$, $\xi(\frac{1}{2} + \frac{iw}{2}) = 0$ (`H_zero_eq_zero_iff_of_H0_eq_xi`). By `riemannXi_eq_zero_iff_strip_zero`, $s := \frac{1}{2} + \frac{iw}{2}$ is a zero of $\zeta$. By the half-plane hypothesis, $\operatorname{Re} s \le \theta$. By the elementary identity `xiArg_re`, $\operatorname{Re} s = \frac{1}{2} - \frac{\operatorname{Im} w}{2}$. Hence

$$-\operatorname{Im} w \;\le\; 2\theta - 1.$$

This is `DBN.H0_neg_im_le_of_zeta_halfplane`. Now $H_0$ is even, `H_neg`: $H_t(-z) = H_t(z)$, because the integrand depends on $z$ only through $\cos(zu)$. So $-w$ is also a zero and the same bound gives $\operatorname{Im} w \le 2\theta - 1$. Together,

$$(\operatorname{Im} w)^2 \;\le\; (2\theta - 1)^2,$$

which is `DBN.H0_im_sq_le_of_zeta_halfplane`; the step from the two one-sided bounds to the square is `nlinarith`. Finally apply `m1_H_real_zeros_of_im_sq_le` with $t_0 = 0$ and $Y = (2\theta - 1)^2$: a zero of $H_t$ is real as soon as $Y \le 2t$, that is $t \ge (2\theta - 1)^2/2 = 2(\theta - \frac{1}{2})^2$. The hypothesis $0 \le t$ that the heat-flow lemma needs follows from the same inequality since a square is nonnegative. The instance $\theta = 7/8$ is `norm_num` and `linarith`: $(2 \cdot 7/8 - 1)^2/2 = (3/4)^2/2 = 9/32$.

Nothing here is deep. The depth is in `dbn_debruijn_parametric` and in OpenAI's theorem, and both are inherited.

## 4 The seam, and its removal

### 4.1 Two revisions

The `dbn` island is a Lake package pinned to Lean v4.34.0-rc1 and Mathlib revision `de5ce8a9`. OpenAI's library is pinned to Lean v4.34.1 and Mathlib revision `d13f23b`, about six weeks later. Both revisions declare the constant `riemannZeta`, and the statement

$$\forall\ \texttt{s} :\ \mathbb{C},\ \texttt{(7 / 8 :}\ \mathbb{R}\texttt{) < s.re} \to \texttt{riemannZeta s} \neq \texttt{0}$$

elaborates at both to an expression with the same constants, the same instance terms and the same numerals. Our first construction, on October 7, was the obvious one: prove `dbn_real_zeros_of_qrh` on the island, prove `qrh_seven_eighths` in OpenAI's workspace, and observe that the hypothesis of the former is the conclusion of the latter. That observation is true as text. The question is whether it is true as mathematics, and whether a kernel can be made to check it.

### 4.2 The fidelity analysis

We answered the first question with lean4export [19]. In each environment we copied the elaborated statement into a definition `QRHFidelity.stmt` (on the island side, the hypothesis of `dbn_real_zeros_of_qrh`; on OpenAI's side, the type of `qrh_seven_eighths`) and exported it with its full dependency closure. The island side used lean4export built at tag v4.34.0-rc1; OpenAI's side used Comparator's copy at v4.34.1. A script (`lean/fidelity/compare_exports.py` in [15]) then hashed every constant structurally and walked the *meaning closure* of the statement: the types of everything, the values of definitions, opaques and recursor rules, but not the proofs of theorems, which are irrelevant by proof irrelevance. It normalized three sources of noise: universe-parameter renaming; hygienic, private and auxiliary names, which embed module paths and so change when a file moves; and references to theorems, which are compared by statement.

| | island side | OpenAI side |
|---|---|---|
| Lean | v4.34.0-rc1 | v4.34.1 |
| Mathlib | `de5ce8a9` | `d13f23b` |
| constants exported | 53,251 | 53,309 |
| constants in the meaning closure of the statement | 4,978 | 4,982 |
| normalized digest of the statement | identical | |
| named definitions in the closure that differ | 267 | |
| theorem statements in the closure that differ | 87 | |
| root-cause definitions (differ, dependencies agree) | 27 | |

**Table 1:** The cross-revision comparison of `riemannZeta s` $\neq$ `0`. Data: `lean/fidelity/RESULT_2026-10-07.json`.

Table 1 summarizes the result. The statement itself is identical after normalization. Beneath it, 267 named definitions differ. Twenty-seven of them are root causes, definitions whose own named dependencies agree but whose bodies do not, and they fall into recognizable groups: Mathlib's refactor of the algebraic hierarchy (`AddMonoid.mk`, `Monoid.mk` and their siblings take different parents in a different order, and every projection changes with them); `WellFoundedLT` is `IsWellFounded` at one revision and `WellFounded` at the other; `MeasureTheory.eLpNorm` gained a `[TopologicalSpace]` argument, and $\zeta$ is defined through Mellin-transform integrals so measure theory is in its closure; core `Bool.and` is defined through a `match` auxiliary at rc1 and directly through `Bool.rec` at v4.34.1; and a dozen private or auxiliary constants (`Real.instMax`, `Nat.findX`, `ENNReal.instLE` and others) changed content.

None of this is surprising, and none of it is wrong. Every one of these refactors is meant to preserve the mathematics, and $\zeta$ is pinned down by its Dirichlet series on $\operatorname{Re} s > 1$, its holomorphy on $\mathbb{C} \setminus \{1\}$ and the identity theorem, so the two `riemannZeta`s are the same function by a standard argument. But that argument is not machine-checked, and with these two revisions it cannot be: no single kernel ever sees both closed terms. A grant that plugged the OpenAI theorem into the island corollary across the seam would have been, in the words we used in the registry, a mathematical identification and not a kernel check. The registry's design permits cross-island grants,

and has one precedent; but that precedent was cross-*island* at a common Mathlib, not demonstrably cross-*meaning*. We judged the honest course to be to say so in words wherever the cross-revision pairing appeared, and then to remove the seam.

### 4.3 The single-pin port

The removal is mechanical in principle and was mechanical in practice. The import closure of `DBNZeroFreeHalfplane` on the island is nineteen modules: the definitions, the $\xi$ representation and its integration by parts, the Hadamard product and its count, mean and linear-growth lemmas, the heat-flow step, the Hurwitz argument, the parametric theorem, and the corollary. Together they are 5,512 lines. They import two modules of the LiCriterion library [16] for $\xi$ as an entire function: `Lc/XiZeros.lean`, which we copied unmodified, and `Lc/LiCriterion/Basic.lean`, a 5,159-line file of which the closure uses exactly three declarations (`NontrivialZero`, `riemannXi` and `xi_zeros_are_nontrivial_zeros`); we copied those three verbatim with the file's `open` lines and namespace into a trimmed shim, so that the module name, and hence every `import` in the nineteen modules, stays the same, and we recorded the trimming as a modification under the library's Apache-2.0 licence. The upstream file also carries literature axioms that no `dbn` theorem used; the island's axiom guard was already clean of them, and the shim does not include them.

We copied all of it into OpenAI's workspace and built. Seventeen of the nineteen modules compiled unchanged, byte for byte (checked with `cmp`). Two did not:

- `DBNHadamardProduct.lean`, line 207: `Finset.prod_le_prod` became `Finset.prod_le_prod₀`. Mathlib renamed the lemma for products of nonnegative factors on September 1, 2026, and gave the old name to a different lemma (the ordered-monoid version, formerly primed) with one argument fewer.

- `DBNHadamardMean.lean`, line 116: the hypothesis of `Real.posLog_le_posLog` changed from $0 \le x$ to $-1 \le x$. The weaker hypothesis is derived from the one the proof already had: `(norm_nonneg _)` became `(neg_one_lt_zero.le.trans (norm_nonneg _))`.

Both are repairs inside proofs. No statement and no definition changed. The two patches are recorded as unified diffs in `lean/patches/`, with the reason for each, and `scripts/materialize.sh` applies them after copying the modules, so that the repository carries the island sources untouched and the change is visible. Two build options were restored to match each source package: the island builds with Lean's default `autoImplicit := true`, OpenAI's package sets it `false`, and the LiCriterion shim keeps its upstream options.

After the port, `dbn_real_zeros_of_qrh qrh_seven_eighths` typechecks in one environment, and the seam is gone for this theorem. The island's own artifacts at v4.34.0-rc1 are unchanged; they still pair with the bridge only across revisions, and the registry's statement of the bridge node still says so (Section 5.5).

### 4.4 Why not a Lake dependency

A reader who knows Lake will ask why the bridge is a recipe rather than a package that requires OpenAI's library. We tried that first. `require OAI from git "https://github.com/openai/math.git" @ "adc7f124" / "lean"` runs `lake update` for six minutes and then fails: OpenAI's lakefile patches twenty-three of its dependencies for Lean v4.34.1 compatibility, from a `run_cmd` during lakefile elaboration and from a `post_update` hook, and both assume that OpenAI's package is the workspace root. The hook refuses a checkout under a downstream `.lake/packages`. Vendoring

the import closure of Theorem 2, about 2,924 modules and 486,000 lines plus parts of two further libraries, would have made us the maintainers of half a million lines we did not write. So `materialize.sh` checks out OpenAI's `lean/` directory unmodified at the pinned commit, verifies the toolchain and Mathlib pin, copies our files into its root, applies the two patches, and appends our `lean_lib` declarations to OpenAI's lakefile. OpenAI stays the root; their patches apply as intended; our modules build inside. On a laptop, starting from an already-built copy of their library, the two bridge modules compile in 45 seconds; the full port compiles in 94 seconds of wall time with the OpenAI closure replayed from cache. On a GitHub-hosted runner with no cache the whole workspace builds in about 80 minutes.

# 5 Verification

We describe the checks in the order in which they add independence from the authors.

## 5.1 The kernel, and the axiom guards

Every theorem in this paper was checked by Lean's kernel when its module compiled. Two guard modules, `AxiomGuardQRHBridge.lean` and `AxiomGuardDBNUnconditional.lean`, run `#print axioms` on Theorem 1, on Theorem 2 in both its OpenAI and re-exported forms, on the corollary and its conditional instance, on de Bruijn's theorem, on the $\xi$ representation, on LiCriterion's $\xi$-zeros theorem, and on the evenness of $H$. All print `[propext, Classical.choice, Quot.sound]`; none prints `sorryAx`. The continuous-integration job greps for both facts and fails otherwise.

## 5.2 The second kernel: Comparator and nanoda

openai/ten-proofs Comparator [17] is a checker for formal challenges. Given a *challenge* module, which states a theorem with a `sorry` body, and a *solution* module, which proves it, Comparator exports both environments with lean4export, checks that every constant the challenge declares has exactly the same definition in the solution, checks that the solution's proof uses only a whitelist of axioms, and replays the proof through Lean's kernel and, when asked, through nanoda [18], an independently written kernel in Rust. A solution that proves a different theorem, or that redefines a challenge constant to make the theorem easier, is rejected at the constant comparison; a solution that smuggles in an axiom is rejected at the whitelist; a proof term the second kernel does not accept is rejected at replay.

We ran Comparator, with nanoda on, on three configurations, all inside the materialized workspace:

1. **The unconditional theorem.** The challenge `ArdaDBNChallenge.lean` imports only Mathlib. It transcribes the definitions of `DBN.thetaMoment`, `DBN.Φ`, `DBN.HIntegrand` and `DBN.H` verbatim from the island's `DBNDefs.lean`, under that file's own `open` line and namespace, and then states Theorem 1 with `sorry`. The solution is `ArdaDBNUnconditional`. Comparator's constant comparison therefore forces the solution's `DBN.H` to be exactly the transcribed term: the solution cannot be proving the theorem about some other function called $H$. Verdict: "nanoda kernel accepts the solution. Lean default kernel accepts the solution. Your solution is okay!" In 473 seconds and 9.1 GB of memory on the author's machine; 760 seconds in the independent audit; 29 minutes for the whole Comparator stage on the CI runner.

2. **The negative control.** The same challenge with one character changed: the exponent $e^{9u}$ in $\Phi$ becomes $e^{8u}$. Verdict: "`uncaught exception: Const does not match between challenge`

`and target 'DBN.Φ'`", exit 1. The judge discriminates. The CI job requires this rejection and fails if the tampered challenge is ever accepted.

3. **The half-plane re-export.** `qrh_seven_eighths` in self-check mode (challenge module equal to solution module), which exercises the axiom whitelist and both kernel replays over OpenAI's entire proof. Verdict: accepted, 516 seconds, 9.4 GB. The Mathlib-only reading of OpenAI's statement had already been checked by our replay of their own challenge file `ComparatorChallenges/QuasiRiemannHypothesis.lean`, which imports only Mathlib and states Theorem 2 with `sorry`; that replay, and the replay of the three other family-003 challenges, was done on October 7 before any of this work began, and both kernels accepted all four.

One footgun deserves recording for anyone who writes such a challenge. The first run of configuration 1 was rejected with "Const does not match" on `DBN.Φ` even though the text of Φ was identical. Under `pp.all` the only difference was the auxiliary proof of the `Nat.AtLeastTwo 4` instance inside Φ. In `DBNDefs.lean`, `thetaMoment` is elaborated first; Lean abstracts that instance proof into `DBN.thetaMoment._proof_1` and reuses it in Φ. A challenge without `thetaMoment` generates `DBN.Φ._proof_5` instead. Transcribing `thetaMoment` too, although $H$ never mentions it, made the `pp.all` dumps identical and the run accept. So a transcribed challenge must reproduce not only the definitions the statement mentions but the earlier definitions whose auxiliary lemmas those definitions reuse.

Comparator's own sandbox, landrun, is Linux-only; the local runs used a pass-through shim and are therefore not sandboxed, which the logs say. The CI runs use the same shim on Linux for consistency with the project's other judge jobs.

### 5.3 The independent audit

On October 7, after the branch was pushed, a second session with no part in writing the proofs audited it from a fresh worktree of the pushed branch, without reading the authoring session's working directory. The audit [15] is a table of claims against checks. The checks that matter most:

- OpenAI's theorem statement, toolchain file, Mathlib pin and licence were fetched from GitHub at commit `adc7f124`, not from any local copy, and compared with the bridge and with `OAI_PIN`.
- The challenge's transcription of $H$ was diffed against `DBNDefs.lean` and against the registry's vocabulary mirror; identical up to documentation comments.
- The nineteen-module closure was recomputed from the island sources; the ported files were diffed against the island and found byte-identical except for exactly the two recorded patches; the LiCriterion shim was diffed against the upstream repository at its pinned commit.
- The bridge, port, theorem and both challenges were built from source (11,900 jobs; the 27 project modules compiled rather than replayed from the authoring session's cache), the island corollary was recompiled from source on the island, and all axiom guards were re-run.
- All three Comparator configurations were re-run with the auditor's own fresh builds of Comparator and nanoda. Same three verdicts.

The audit also recorded a limitation that shaped what came next: every session on the machine commits under one git identity, so no session there could satisfy the registry's independence rule for a read-back. It is this limitation that the registry step was designed to expose.

### 5.4 Continuous integration

The Arda repository's workflow `oai-qrh-bridge.yml` [14] runs on every push that touches the island: it frees disk, materializes OpenAI's checkout cold, resolves their dependencies, fetches Mathlib's oleans, builds the five modules, saves a checkpoint even if the build fails so the next run resumes, re-elaborates the two committed artifact files against the built workspace (this is what the registry's coverage gate attributes to the artifacts), runs the axiom guards, builds Comparator from tag v4.34.0 with its toolchain file set to v4.34.1 (there is no v4.34.1 tag) and nanoda from source, and runs the three Comparator configurations with the rejection of the tampered control required. Its first run failed for a reason worth recording: the re-elaboration step runs in a directory with no `lean-toolchain` file, and the runner has no elan default, so the step died with "no default toolchain configured" after the eighty-minute build had succeeded. Pinning `ELAN_TOOLCHAIN` for that step fixed it, and the rerun, restoring the checkpoint, went green in about 35 minutes end to end. The accompanying repository's `verify.yml` is the same recipe, run from this repository's own files.

### 5.5 The registry

The Arda project tracks formal results in a registry of nodes, each a TOML file with a verbatim Lean statement, a status, a proof link to an artifact, and a provenance block. A node is `draft` when registered; it becomes `open` only when an independent read-back of its formal statement is recorded; it becomes `proved` only when a verify gate confirms that the artifact declares exactly the registered statement and then proves it, that a runnable CI step compiles the artifact, and that the dependencies a reduction needs are already proved. Independence of a read-back is enforced by the tool: it refuses an auditor whose session or identity matches the node's registering author. Independence can also come from a second route: a separate CI job, the *judge*, renders each proved node's registered statement as the type of a bridge theorem whose proof is the artifact's constant, builds it (which succeeds only if the artifact proves exactly the registered proposition), and runs Comparator with both kernels on it; a pass is recorded on the node with the verdict read from the job log and the artifact hashed at the judged commit.

Four nodes were registered on October 7: the corollary for general $\theta$, its instance at $7/8$, the half-plane re-export, and Theorem 1. Three things happened before any of them was granted, and the order matters.

First, the read-backs. The authoring session had written the proofs and a second session had registered the nodes, so neither could record a read-back, and every session on the machine shared one git identity in any case. The project's design allows the operator to be the renderer. On October 9 the operator wrote the four read-backs in his own words under his own identity and session label; the tool accepted them as independent.

Second, the grants. The verify gate flipped all four nodes to `proved`, recording the SHA-256 of each artifact and each statement file.

Third, the judge. The registry's judge could not consume the bridge island as it stood, because the island is a recipe rather than a Lake package that the judge could path-require; OpenAI's library has to be the workspace root. An extension (`MATERIALIZED_ROOT_ISLANDS`) taught the judge to install its bridge modules inside the materialized workspace as an appended `lean_lib`, and a new CI job materializes the workspace, builds the bridge modules by name and runs Comparator on them. The extension was reviewed by the session that had audited the proofs, which verified among other things that the six pre-existing judge bundles were byte-identical after the change and that the two generated bridge theorems, with the registry's statement text as their type, were accepted

by both kernels in its own workspace. The judge run on the granted registry passed all four nodes: the two island nodes in the existing `dbn` shard, the two bridge nodes in the new job, each with nanoda. The records were written with `log_check = "verified"` and `head_check = "matched"`.

The registered statements that the judge used as types are in `provenance/registry/` of [15], and the generated bridge theorems in `lean/judge/`. For Theorem 1 the bridge is

```
theorem MissionJudge.RH_dbn_real_zeros_nine_thirtyseconds :
    ∀ t : ℝ, 9 / 32 ≤ t → ∀ z : ℂ, DBN.H t z = 0 → z.im = 0 :=
  dbn_real_zeros_of_qrh_unconditional
```

imported together with the island's two axiom guards, so that any constant of the statement's vocabulary re-declared by the artifact would be a duplicate-declaration error rather than a silent substitution. For these two nodes the vocabulary is `DBN.H`, declared inside the guards' closure, and Mathlib's `riemannZeta`, both written at the root with no `open` context, so neither can be redirected.

### 5.6 What is checked and what is trusted

| **Kernel-checked (Lean and nanoda)** | **Trusted as code or as definition** |
|---|---|
| Theorem 1, its proof term, and every ancestor: Theorem 4, Theorem 3, $H_0 = \xi/8$, the $\xi$-zeros theorem, OpenAI's Theorem 2 and its entire closure, all under three standard axioms | Mathlib's definitions of `riemannZeta`, `completedRiemannZeta`$_0$, the Bochner integral and the reals, as the meaning of the symbols |
| That the challenge's transcribed $H_t$ is the $H_t$ of the theorem (Comparator constant comparison) | That the transcribed $H_t$ is the Polymath15 $H_t$ of Equation (1) (by inspection of eleven lines) |
| That the proof uses no axiom outside the whitelist | OpenAI's pinned tree at `adc7f124`, verified byte-for-byte against the commit and never modified; about 2,900 modules outside the island guards' import closure |
| That the registered statement is the type of the artifact's theorem (the judge's bridge module builds) | Lean's kernel, nanoda, lean4export, Comparator, and the toolchain binaries |
| That the island modules compiled at OpenAI's revision are the island modules (byte identity plus two recorded patches) | That the registry's read-backs describe the statements correctly (the operator's rendering; and the judge, which settles proof-matches-statement but not statement-matches-title) |

**Table 2:** The trust boundary for Theorem 1.

Table 2 is the honest boundary. Two entries on the right deserve a sentence each. The pinned OpenAI tree is trusted in the sense that we did not read 486,000 lines; we verified that the tree is the commit it claims to be, that the kernel accepts it, that a second kernel accepts it, and that the statement at its top is the one in their challenge file. That is the same trust anyone who cites Theorem 2 extends, and it is more than most citations get. And the definition of $\zeta$ is trusted in the sense that a formal theorem about `riemannZeta` is a theorem about Mathlib's object; that Mathlib's object is Riemann's function is a matter of Mathlib's own documentation and its proofs of the Dirichlet series, the functional equation and the Euler product, which are in its library.

## 6 Version 1.1: Liu's half-plane, and the end of this route

On 8 October 2026, three days after OpenAI's release, B. Liu posted an improvement of its half-plane [11]: following the argument of [12] and varying two of its length parameters, $b$ and the prime-slot length $\ell$, he obtains

$$\zeta(s) \neq 0 \qquad \text{for} \qquad \operatorname{Re} s > B := \frac{1507 - 2\sqrt{921}}{1653} = 0.87495706979\ldots,$$

at $\ell = (33 + 8\sqrt{921})/1653$, a hair above OpenAI's $1/6$, together with a Lean formalization at OpenAI's exact pin. The converter of Theorem 4 turns it into

**Theorem 6** (`dbn_real_zeros_of_liu_unconditional`, `dbn_real_zeros_ge_2811857`)**.** *For every real $t \geq 2(B - \frac{1}{2})^2 = (1867057 - 10888\sqrt{921})/5464818 = 0.28118560838\ldots$ and every $z \in \mathbb{C}$, $H_t(z) = 0$ implies $\operatorname{Im} z = 0$. In particular every zero of $H_t$ is real for $t \geq 0.2811857$; classically $\Lambda \leq 0.2811857$.*

**How it was checked.** The composition lives in one Lean workspace: an unmodified openai/math at `adc7f124`, Liu's sources at commit `7d10420` placed into it as an overlay, the single-pin dbn port of Section 4, and our bridge, which re-exports Liu's `CombinedPaper.algebraic_zeta_nonzero` as `liu_zeta_zero_free` (the case $s = 1$ by Mathlib's `riemannZeta_one_ne_zero`; an independent replay found that Liu's own hypothesis $s \neq 1$ is in fact unused) and applies the converter. The axiom guard prints `[propext, Classical.choice, Quot.sound]` for each of its fourteen lines. Liu declares 465 names inside OpenAI's `OAI` namespace; a check against all 3232 compiled OpenAI modules found no clash, and in one workspace any clash would be a duplicate-declaration error rather than silent shadowing. Liu's upload did not build as published (the vendored OpenAI closure was absent and the module tree had been flattened by GitHub's upload limit); an independent replay reconstructed both from hash-verified copies, built it (12,088 jobs) and confirmed the standard axioms. Comparator with nanoda accepted the composed theorem against a challenge importing only Mathlib and rejected the tampered control, in CI and in the registry judge, which replayed all three registered statements (run 38179118143). The three registry nodes were read back by the author of this paper before they were granted, as for the four nodes of Section 5.5.

**Provenance and licence.** Neither Liu's nor OpenAI's code is redistributed: the recipe fetches both at pinned commits and verifies every file against recorded hashes. Liu's repository carries no licence file; for this work it is treated as Apache-2.0 by decision of the repository owner (11 October 2026), with no grant from the author on file, attribution kept, and a licence requested from him.

**Why this is the end of the route.** An independent model of the Stage II exponent system of [12], built by the Arda team before Liu's paper appeared, located the optimum of the whole architecture with proved inputs at $\sigma_0 = 0.87495707$, where the high-side witness-count constraint crosses the low-side law $\sigma_0 = 11/12 - \ell/4$; the binding bin sits at $\delta = 49/48 - \sqrt{921}/48$, which is where Liu's $\sqrt{921}$ comes from. The method's limit and Liu's theorem coincide to eight digits. Pushing the half-plane further along this route would require replacing the sextic structure of the argument, which is forced by the existence of explicit cubic theta coefficients; that is a question about metaplectic forms, not about parameters. Theorem 6 is therefore the best bound on $\Lambda$ that OpenAI's method can deliver, and it is still weaker than the numerical 0.22 and 0.2.

## 7 What this does and does not advance

**On $\Lambda$.** Theorem 1 does not improve the known upper bound on $\Lambda$. The ladder stands at 0.2 [10], and $9/32 = 0.28125$ is above it, above Polymath15's 0.22, and below only de Bruijn's $1/2$. What is new is the kind of certificate: $9/32$ is a theorem whose entire proof, from the definition of $H_t$ through OpenAI's zero-free half-plane to the final application, has been replayed by two proof kernels; 0.22 and 0.2 rest on rigorous but unformalized computation. We do not suggest that this makes $9/32$ a better bound. It is a worse bound with a different warranty.

**On the route.** Theorem 4 is a verified conversion from zero-free half-planes to bounds on $\Lambda$, and it is lossy in a way worth making explicit. It gives $\Lambda \le 2(\theta - \frac{1}{2})^2$, so it beats the Polymath15 bound exactly when

$$2\big(\theta - \tfrac{1}{2}\big)^2 < 0.22 \iff \theta < \tfrac{1}{2} + \sqrt{0.11} \approx 0.8317,$$

and it beats Platt–Trudgian when $\theta < \frac{1}{2} + \sqrt{0.1} \approx 0.8162$. A zero-free half-plane at 0.83 would therefore set a new, kernel-checked record on $\Lambda$ with no further work: the registry node for the general-$\theta$ corollary is proved, and a new half-plane theorem drops into it as a new instance. OpenAI's $7/8 = 0.875$ is close to that threshold and on the wrong side of it. Whether the methods of [12] can be pushed from $7/8$ toward 0.83 is a question for the authors of that work, not for this paper.

**On the Riemann Hypothesis.** Nothing here bears on it. A half-plane at $\frac{1}{2} + \varepsilon$ gives $\Lambda \le 2\varepsilon^2$ through this route, and $\Lambda = 0$ requires $\varepsilon = 0$, which is the Riemann Hypothesis itself. The conversion cannot close the gap; at best it transports progress on the half-plane into progress on $\Lambda$, with a quadratic loss. The registry marks every one of the four nodes, and this paper marks every section, with the sentence that the project has carried on every artifact since its first day: `conjecture1_proved = False`.

**On method.** Two things from the exercise seem to us to be of use beyond it. The first is the fidelity analysis of Section 4: when two formal results are to be composed across library revisions, exporting both and walking the meaning closure of the shared statement is cheap, mechanical, and decisive about whether a kernel can check the composition or whether a human must assert it. We expect this situation to become common as formal libraries are released by many hands at many pins. The second is the governance of Section 5.5. The useful part of it was not the tooling but the refusal built into the tooling: the read-back command would not accept a self-audit, and the judge would not record a pass on a node that was not yet granted. Those refusals forced the order read-back, grant, judge, record, and forced a person other than the authoring sessions into the loop. A result that enters the record that way carries its own provenance.

## Acknowledgements

The theorem $\zeta(s) \neq 0$ for $\operatorname{Re} s > 7/8$ is OpenAI's [12, 13], released under the Apache License 2.0; this paper adds one line to it and proves nothing about $\zeta$. The de Bruijn heat-flow machinery is the Arda project's [14]; its $\xi$ is LiCriterion's [16]; the second kernel is nanoda [18] through Comparator [17]; everything runs on Lean 4 and Mathlib [20, 21]. The formalization, the audit, the tooling and this paper were produced with extensive use of an AI coding assistant (Claude, Anthropic) under the direction of the author, in several concurrent sessions whose division of labour, including the independent audit and review, is recorded in the accompanying repository. The Lean kernel is the

sole trusted component: every claim in this paper is either a kernel-checked theorem or labelled otherwise.

## References


[1] N. G. de Bruijn, The roots of trigonometric integrals, *Duke Math. J.* 17 (1950), 197–226.

[2] C. M. Newman, Fourier transforms with only real zeros, *Proc. Amer. Math. Soc.* 61 (1976), 245–251.

[3] G. Csordas, T. S. Norfolk and R. S. Varga, A lower bound for the de Bruijn–Newman constant $\Lambda$, *Numer. Math.* 52 (1988), 483–497.

[4] G. Csordas, W. Smith and R. S. Varga, Lehmer pairs of zeros, the de Bruijn–Newman constant $\Lambda$, and the Riemann Hypothesis, *Constr. Approx.* 10 (1994), 107–129.

[5] A. M. Odlyzko, An improved bound for the de Bruijn–Newman constant, *Numer. Algorithms* 25 (2000), 293–303.

[6] Y. Saouter, X. Gourdon and P. Demichel, An improved lower bound for the de Bruijn–Newman constant, *Math. Comp.* 80 (2011), 2281–2287.

[7] H. Ki, Y.-O. Kim and J. Lee, On the de Bruijn–Newman constant, *Adv. Math.* 222 (2009), 281–306.

[8] B. Rodgers and T. Tao, The de Bruijn–Newman constant is non-negative, *Forum Math. Pi* 8 (2020), e6; arXiv:1801.05914.

[9] D. H. J. Polymath, Effective approximation of heat flow evolution of the Riemann $\xi$ function, and a new upper bound for the de Bruijn–Newman constant, *Res. Math. Sci.* 6 (2019), 31; arXiv:1904.12438.

[10] D. Platt and T. Trudgian, The Riemann hypothesis is true up to $3 \cdot 10^{12}$, *Bull. London Math. Soc.* 53 (2021), 792–797; arXiv:2004.09765.

[11] B. Liu, Slightly improved zero-free half-planes for the quasi-Riemann hypothesis, arXiv:2610.12234 (2026); Lean formalization at https://github.com/liubaiying101/Slightly-improved-zero-free-half-planes-for-the-quasi-Riemann-hypothesis, commit `7d10420`.

[12] OpenAI, The Quasi-Riemann Hypothesis, OpenAI Math Release preprint `OAI:The-Quasi-Riemann-Hypothesis-October-5-2026` (2026); earlier version `OAI:The-Quasi-Riemann-Hypothesis-September-30-2026`, "A zero-free half-plane $\mathrm{Re}(s) > 7/8$". https://github.com/openai/math/tree/main/preprints.

[13] OpenAI, `openai/math`: Lean formalizations and preprints, commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a` (October 6, 2026), Apache License 2.0. https://github.com/openai/math. The theorem used here is `OAI.riemannZeta_ne_zero_of_seven_eighths_lt_re` in `lean/OAI/NumberTheory/DirichletL/Nonvanishing.lean`; its Comparator challenge is `lean/ComparatorChallenges/QuasiRiemannHypothesis.lean`.

[14] P. W. Murphy, Arda: the Telperion engine and the missions registry, https://github.com/DrMurphyIsIn/Arda. The `dbn` island is `telperion/examples/dbn/lean`; the bridge island is `telperion/examples/oai_qrh_bridge`; the registry nodes are `RH_dbn_real_zeros_of_zeta_halfplane`, `RH_dbn_real_zeros_of_qrh`, `RH_zeta_zero_free_seven_eighths` and `RH_dbn_real_zeros_nine_thirtyseconds` in `telperion/missions/rh`.

[15] P. W. Murphy, qrh-debruijn-newman: Lean sources, scripts, audit records and this paper, https://github.com/DrMurphyIsIn/qrh-debruijn-newman, v1.0.0 (2026), archived at Zenodo, https://doi.org/10.5281/zenodo.23269135 (this version) and https://doi.org/10.5281/zenodo.23269134 (all versions).

[16] N. Bulka, li-criterion-rh-equivalence-lean, commit 35df682f, Apache License 2.0. https://github.com/nicholasbulka/li-criterion-rh-equivalence-lean.

[17] Lean FRO and OpenAI, comparator: a checker for formal challenges (openai/ten-proofs Comparator), tag v4.34.0. https://github.com/leanprover/comparator.

[18] C. Kernan (ammkrn), nanoda_lib: a type checker for Lean 4 export files, in Rust. https://github.com/ammkrn/nanoda_lib.

[19] lean4export: an exporter for Lean 4 environments. https://github.com/leanprover/lean4export.

[20] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction – CADE 28*, LNCS 12699, Springer (2021), 625–635.

[21] The mathlib Community, The Lean mathematical library, in *CPP 2020*, ACM (2020), 367–381. Revisions de5ce8a9 and d13f23b723b8a846827a245b89c10fc7d3f11612 (2026).

# A Reproduction

The repository [15] is self-contained apart from Lean, Mathlib and OpenAI's library, which it fetches at pinned commits. From a clean clone:

```
scripts/materialize.sh          # openai/math lean/ at adc7f124, our files copied in
cd work/oai
lake update && lake exe cache get
lake build ArdaQRHBridge AxiomGuardQRHBridge ArdaDBNUnconditional \
           ArdaDBNChallenge ArdaDBNNegativeControl
lake env lean AxiomGuardDBNUnconditional.lean   # 3 axioms, no sorryAx
```

The first build compiles OpenAI's import closure, about 2,924 modules and 11,900 Lake jobs, and needs about 25 GB of disk and several CPU-hours (about 80 minutes on a 4-core GitHub runner). For the Comparator stage, build leanprover/comparator at tag v4.34.0 with its lean-toolchain set to leanprover/lean4:v4.34.1, build nanoda_lib with cargo build --release, point COMPARATOR_LEAN4EXPORT and COMPARATOR_NANODA at the binaries (and COMPARATOR_LANDRUN at a pass-through shim where landrun is unavailable), and run from work/oai:

```
lake env comparator dbn_real_zeros_of_qrh_unconditional.comparator.json # accept
lake env comparator qrh_seven_eighths.comparator.json                   # accept
lake env comparator tampered.comparator.json                            # REJECT
lake build MissionChallenges.RH_dbn_real_zeros_nine_thirtyseconds \
           MissionChallenges.RH_zeta_zero_free_seven_eighths
lake env comparator RH_dbn_real_zeros_nine_thirtyseconds.comparator.json
lake env comparator RH_zeta_zero_free_seven_eighths.comparator.json      # registry form
```

Each positive configuration prints "Your solution is okay!"; the tampered one fails with "Const does not match between challenge and target 'DBN.Φ"'. Each Comparator run takes eight to thirteen minutes and about 9 GB of memory. The workflow .github/workflows/verify.yml is this recipe, verbatim.

To re-run the cross-revision comparison of Section 4, `lean/fidelity/run_fidelity.sh` needs a built checkout of the Arda `dbn` island at its own revision in addition to the materialized workspace; its output on October 7 is `lean/fidelity/RESULT_2026-10-07.json` and `root_causes_2026-10-07.txt`.

## B The verdicts, verbatim

Local, author's machine, October 7, 2026 (`logs/comparator_local_2026-10-07.txt`):

```
## comparator_qrh_seven_eighths
nanoda kernel accepts the solution
Lean default kernel accepts the solution
Your solution is okay!
      516.18 real       612.10 user        25.56 sys
          9446866944  maximum resident set size
EXIT 0

## comparator_unconditional
nanoda kernel accepts the solution
Lean default kernel accepts the solution
Your solution is okay!
      473.46 real       526.55 user        31.98 sys
          9146286080  maximum resident set size
EXIT 0

## comparator_negative_control
uncaught exception: Const does not match between challenge and target 'DBN.Phi'
EXIT 1
```

Independent audit, second session, own Comparator and nanoda builds, October 7 (`docs/QRH_DBN_AUDIT_2026-10-07_e9.md`): unconditional accepted by both kernels, 760 s; tampered rejected on `DBN.Φ`, 257 s; `qrh_seven_eighths` accepted by both kernels, 758 s.

Continuous integration, Arda run 37728388605, October 8 (`logs/ci_oai_qrh_bridge_run_37728388605.txt`):

```
ACCEPTED: dbn_real_zeros_of_qrh_unconditional
ACCEPTED: qrh_seven_eighths
REJECTED as required: uncaught exception: Const does not match between
  challenge and target 'DBN.Phi'
```

Registry judge, Arda run 37955041118, October 9 (`logs/ci_judge_run_37955041118.txt`):

```
COMPARATOR PASS island=dbn node=RH_dbn_real_zeros_of_zeta_halfplane
    theorem=dbn_real_zeros_of_zeta_halfplane run=37955041118 kernel=nanoda
COMPARATOR PASS island=dbn node=RH_dbn_real_zeros_of_qrh
    theorem=dbn_real_zeros_of_qrh run=37955041118 kernel=nanoda
COMPARATOR PASS island=oai_qrh_bridge node=RH_zeta_zero_free_seven_eighths
    theorem=qrh_seven_eighths run=37955041118 kernel=nanoda
COMPARATOR PASS island=oai_qrh_bridge node=RH_dbn_real_zeros_nine_thirtyseconds
    theorem=dbn_real_zeros_of_qrh_unconditional run=37955041118 kernel=nanoda
```

Axiom guard, every run:

```
'dbn_real_zeros_of_qrh_unconditional' depends on axioms:
    [propext, Classical.choice, Quot.sound]
'dbn_real_zeros_of_qrh' depends on axioms: [propext, Classical.choice, Quot.sound]
'dbn_real_zeros_of_zeta_halfplane' depends on axioms:
    [propext, Classical.choice, Quot.sound]
'qrh_seven_eighths' depends on axioms: [propext, Classical.choice, Quot.sound]
'dbn_debruijn_parametric' depends on axioms: [propext, Classical.choice, Quot.sound]
'dbn_H0_eq_xi' depends on axioms: [propext, Classical.choice, Quot.sound]
'LiCriterion.xi_zeros_are_nontrivial_zeros' depends on axioms:
    [propext, Classical.choice, Quot.sound]
'DBN.H_neg' depends on axioms: [propext, Classical.choice, Quot.sound]
```